AgentProf: Semantic Profiling for AI Agents

Anonymous submission

Abstract

AI agents increasingly orchestrate long-running, multi-step activities across users, models, tools, and system resources. A single run can consume millions of tokens, while production agent deployments already operate within services that handle millions of requests per year and accumulate many trajectories. Improving their quality, safety, and cost efficiency requires finding which task categories consume budget, where failures and wasted work concentrate, and which workflows trigger unsafe effects across runs. This population-level analysis is structurally difficult because the responsible entities are semantic categories rather than code paths, while natural-language behavior and native execution structure do not determine stable identifiers or a reusable cross-run attribution hierarchy. Most existing agent observability tools emphasize per-execution debugging and tracing, and input clustering characterizes distributions rather than attributing aggregate resource consumption to downstream activities. Agent observability needs profiling, not only debugging. Agent profiling requires cross-layer resource projection, stable low-cardinality tags for natural-language behavior, and hierarchical attribution whose granularity can change without reconstructing the data. We present AgentProf, an offline profiler whose semantic operation stack model represents activities as uniform operations, projects them into query-time operation stacks, and uses pluggable *intent attribution* and *stack construction* algorithms to compile local agent trajectories into pprof-compatible profiles. We evaluate AgentProf on 325 real Codex and Claude trajectories containing 183,714 system observations and on public annotated trajectories, using four datasets with 34,539 task-operation instances for localization and nine datasets with 13,265 operations for tag accuracy. Semantic profiling separates over 90% of system-effect cost that per-session views cannot attribute to task categories, locates annotated problems while inspecting 9.4% of work in the top five groups, uses 45% fewer groups than per-session grouping, and derives tags that agree with ground truth on 7 of 9 held-out datasets.

1 Introduction

AI agents (Yang et al. 2024; Anthropic 2025; Xie et al. 2024) orchestrate multi-step activities that span high-level intent, including prompts, large language model (LLM) calls, and tool invocations, as well as low-level system effects such as process spawns, file I/O, and network I/O. Each trajectory interleaves these two layers as the agent acts on a user's request. Agents now undertake substantial projects that span hours, days, or weeks, and a single run can consume millions of tokens; production agent deployments also operate within services that handle millions of requests per year (OpenAI 2026, 2025a). Production teams therefore accumulate many trajectories over weeks or months.

However, improving agent quality, safety, and cost efficiency requires population-level analysis across trajectories and interactions. Developers need to determine which task categories consume the most token budget, where failures and wasted work concentrate across workflows, and which behavioral patterns trigger unsafe system effects. Across many trajectories (Mazaheri and Mazaheri 2026; Lù et al. 2025), answering these questions by manual inspection or per-trajectory LLM evaluation (Zheng et al. 2023) becomes costly.

The structural obstacle is that agent behavior does not automatically supply the profiling entities and hierarchy that code execution supplies. In traditional software, the responsible entities are code paths. Function names are stable, and runtime call nesting provides an attribution hierarchy. For agent problems, the responsible entities must instead be semantic categories such as task intent, workflow phase, and tool operation, and agent trajectories do not automatically provide the two properties that traditional profiling relies on. Prompts are natural language, so two prompts expressing the same intent need not share a common string. Agent executions may expose spans or event order, but native execution structure does not determine a stable cross-run call-stack hierarchy. A sequence of prompts can represent one task or several, and a task can belong to a larger task.

Existing agent observability interfaces generally target per-execution analysis rather than population-level resource attribution. Some (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) organize events into per-execution span trees, which are effective for diagnosing a single run but do not by themselves aggregate resource use by semantic category across runs. Others (Datadog 2025; Laminar 2025) cluster inputs or extract structured signals, but these views characterize input distributions ("30% of users ask code questions") rather than resource attribution

("review tasks consume 40% of the token budget"), because request-level tags do not propagate to downstream tool calls and system effects. Consequently, neither a per-execution tree nor an input-level category alone supplies the aggregate attribution view required by these questions.

Agent observability needs profiling, not only debugging. In traditional systems software, profiling answers population-level questions by attributing aggregate resource consumption to responsible entities rather than inspecting a single execution. We argue that this fundamental methodology transfers to agent trajectories by projecting resources onto responsible entities, assigning stable tags, and attributing the resulting resource measures hierarchically. We therefore propose a *semantic operation stack model* with two core abstractions: (1) uniform *operations* represent prompts, LLM calls, tool invocations, and system effects without type-specific objects, while (2) *operation stacks* replace the runtime call stack with query-time field projections that attribute the same data by task category, execution phase, session, or action type.

An agent profiler must reconstruct three capabilities that code profilers obtain from a call stack. First, cross-layer resource projection must connect system effects to the intent-layer categories responsible for them. Second, intent attribution must derive stable, low-cardinality tags from natural-language behavior so that equivalent activities can be folded across trajectories. Third, stack construction must provide hierarchical attribution at different granularities without treating any single native execution tree as the fixed cross-run hierarchy.

We present AgentProf, an offline profiler that implements this model, compiles local agent trajectories into pprof-compatible profiles (Google 2024b) (Figure 2), and produces population-level views such as Figure 1. Its two pluggable algorithm frameworks (1) derive stable tags through regex rules, a local LLM tagger, or unsupervised clustering and (2) construct operation stacks from user-specified fields or boundaries inferred from visible operation fields. Our evaluation asks four fixed questions: (1) whether semantic profiling improves resource attribution (RQ1), (2) whether profiler output corresponds to real problems (RQ2), (3) how accurate its tags are (RQ3), and (4) what complete profiling costs (RQ4). We evaluate them using 325 real Codex and Claude trajectories with 183,714 system observations and public annotated trajectories. On the real trajectories, semantic profiling separates over 90% of system-effect cost that per-session views cannot attribute to task categories. Across four public datasets with 34,539 task-operation instances, semantic profiling locates annotated problems while inspecting 9.4% of work in the top five groups and uses 45% fewer groups than per-session grouping. Across nine held-out datasets, its derived tags agree with ground truth on seven of nine.

Figure 1 previews the population-level profiles produced by the model developed below.

This paper makes three contributions:

1. **Semantic operation stack model.** We identify the missing profiling layer in agent observability and propose a model of uniform operations and query-time operation stacks (§2–§3).
2. **System implementation with AgentProf.** A profiler that implements this model with pluggable intent attribution and stack construction, producing pprof-compatible output (§4).
3. **Evaluation.** We evaluate resource attribution on 325 real trajectories, localization against hidden ground-truth annotations on four public datasets, tag accuracy on nine held-out datasets, and end-to-end profiling cost (§5). The localization protocol is analogous to injecting known bottlenecks in CPU-profiler evaluation.

2 Background and Motivation

2.1 LLMs and AI Agents

A typical agent trajectory interleaves two layers of activity. At the intent layer, the agent receives a user prompt, issues LLM calls, and invokes tools (code execution, web search, file editing). Each tool invocation triggers lower-layer system effects, including process spawns, file reads and writes, and network requests. A single trajectory may contain hundreds of such intent-effect cycles, and a production team accumulates many trajectories over time.

2.2 System Profiling

Profiling aggregates resource consumption by responsible entity, unlike single-execution debugging and tracing. In traditional software, the profiling pipeline has three steps: (1) a profiler periodically samples execution state, (2) it attaches each sample to the current call stack, and (3) it merges samples with identical stacks. Flame graphs (Gregg 2011) and pprof (Google 2024b) call graphs visualize the result, where width or node size represents aggregate cost. Perfetto (Google 2024a) generalizes this with SQL-based analysis and derived events.

These tools support flexible aggregation, but they all assume stable function names and a runtime call stack. Pprof's `tagroot`/`tagleaf` options can add label-derived pseudo-frames, but only on top of an existing execution stack that agent trajectories do not have.

2.3 Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) record agent events as per-execution span trees that support debugging a single run. AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating eBPF-captured system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Beyond cross-layer resource projection, adapting profiling to agent trajectories adds two core challenges. The first is that the responsible entities differ. In traditional software, profiling attributes cost to code paths, but in agent trajectories the relevant entities are semantic categories such as task intent and workflow phase. Existing span specifications (OpenTelemetry 2024; Arize AI 2024b) do not automatically derive these categories from natural-language prompts. Prompts and tool invocations that express the same intent can differ, so the profiler cannot aggregate them as it aggregates identical function names. The second challenge is that native

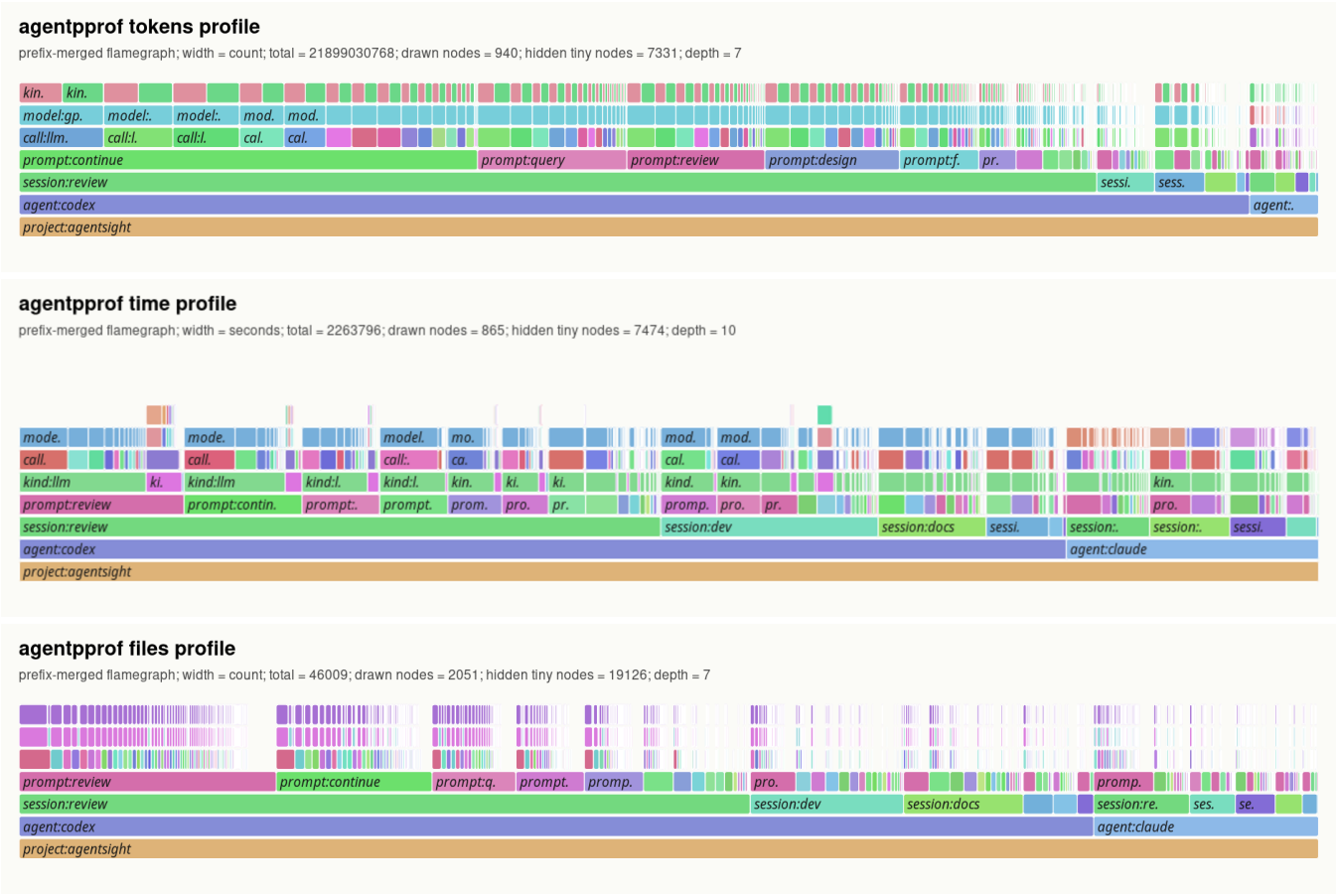

Figure 1: Semantic flame graphs of 325 real agent trajectories under three views. *Top* shows tokens with width representing token count. *Middle* shows time with width representing wall-clock duration. *Bottom* shows files with width representing file-operation count. All three are produced from the same operations by changing only the stack fields and weight function.

execution structure does not determine a reusable cross-run hierarchy for attribution. In CPU profiling, the call stack determines attribution at every level. For example, `malloc`'s cost belongs to `parse`, to `process`, and to `main` simultaneously. A sequence of prompts may constitute one task or several, and a task may be part of a larger workflow. No runtime mechanism, however, marks these boundaries or determines the granularity at which cost should be attributed. We quantify how the missing reusable hierarchy affects resource attribution in RQ1 (§5).

3 Design

3.1 Design Requirements for Agent Profiling

Traditional profiling relies on three mechanisms that the call stack provides automatically. Agent profiling needs all three but must achieve them without a call stack:

DR1: Cross-layer resource projection. In traditional profiling, each sample is taken inside a call context, so resource consumption is automatically attributed to the code path that caused it. Agent activities span two layers, namely intent (prompts, LLM calls) and system effects (file I/O, process spawns). The profiler must connect system-layer resource consumption to intent-layer semantic categories.

DR2: Stable tags for folding. In traditional profiling, function names are stable identifiers that enable folding identical stacks. Agent prompts are natural language, so the profiler must derive short, stable, repeatable tags before folding.

DR3: Hierarchical attribution. In traditional profiling, the runtime call stack provides the attribution hierarchy. Function calls nest, and folding identical stacks produces the tree that profiles visualize. Native event nesting does not supply the reusable semantic hierarchy, so the profiler must construct attribution hierarchies from the data.

Cross-layer resource projection, stable tags, and hierarchical attribution drive the design. Operations (§3.2) satisfy DR1 by representing intent and system-effect activities in a single record with tag inheritance, so intent-layer tags propagate to the system effects they trigger. *Intent attribution* (§3.3) satisfies DR2 by deriving stable tags from natural-language fields. *Operation stacks* (§3.2) satisfy DR3 by replacing the runtime call stack with query-time field projections that attribute resources at multiple levels of granularity, so the same

data supports multiple views without rebuilding the input.

The profiling pipeline has four stages: (1) parse raw trajectories into operations, (2) enrich operations via intent attribution or mapping rules, (3) project each operation onto a stack frame sequence chosen at query time, and (4) fold operations with identical stacks, summing their weights.

3.2 Semantic Operation Stack Model

Because agent activities span both the intent and system-effect layers described in §2, a profiler should not distinguish between them in the data representation. AgentProf therefore represents every activity with one abstraction. An `operation` is a weighted record with string fields and additive measures. Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, ...) and additive measures (token count, duration, event count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operations with the same schema, distinguished only by which fields carry values.

An `operation stack` provides hierarchical attribution for agent trajectories just as a runtime call stack does for code. An ordered list of fields $[f_1, f_2, \ldots, f_k]$ projects each operation o to a frame sequence $\langle o.f_1, o.f_2, \ldots, o.f_k \rangle$, and operations with identical sequences are merged, summing their weights. Unlike a call stack, the fields are chosen at query time, not determined by execution.

The same operations can therefore produce a flat task summary, a per-session tree, a semantic phase/action hierarchy, or a dataset-native hierarchy. Changing the fields changes attribution without changing the data. Sessions and spans are optional operation fields, not fixed hierarchy levels, so the same data supports both debugging views (include `session`) and aggregate profiling views (exclude `session`).

A *profile view* is a triple (φ, σ, w) in which a predicate φ selects which operations participate, a stack function $\sigma = [f_1, \ldots, f_k]$ maps each operation to its frame sequence, and a weight function w assigns a nonnegative measure. Here, attribution is an accounting procedure that assigns a conserved additive measure to a declared responsibility path but does not by itself establish a causal diagnosis. Four built-in views cover common questions: (1) `tokens` weights LLM calls by token count, (2) `time` weights all timed operations by duration, (3) `files` weights file operations by event count, and (4) `network` weights network operations by event count.

3.3 Intent Attribution and Stack Construction

Intent attribution. Intent attribution maps natural-language prompts to short, stable, repeatable tags, giving agent trajectories the stable identifiers that function names provide in traditional profiling (§2). The framework supports three pluggable backends: (1) ordered regex rules, (2) a local LLM tagger, and (3) unsupervised clustering for rule authoring. For structured trajectories where action types and quality annotations are explicit fields, mapping rules derive new fields from existing ones (e.g., `phase:navigate=type:(click|scroll|key)`).

Stack construction. Stack construction builds attribution hierarchies from available fields. When users supply an ordered list of fields, the projection is direct. When no field list is supplied, AgentProf induces operation stacks automatically. It scores candidate boundaries using changes in visible fields and token-set distance between adjacent operations, then recursively selects splits that improve within-segment field consistency subject to balance and depth constraints. The induced stacks have variable depth, and a configurable depth cap controls the granularity. This mode produces coarser groups than hand-specified stacks but requires no domain knowledge, making it useful for initial exploration (§5).

4 Implementation

Inputs and operation reconstruction. AgentProf is an offline profiler implemented as a Rust CLI and parser (∼9.8K LOC, Figure 2). It reads Codex (OpenAI 2025b) and Claude Code (Anthropic 2025) local JSONL history files and AgentSight (Zheng et al. 2025) recordings for system effects, and reconstructs prompts, LLM calls, tool invocations, and their triggered file and network effects.

Tagging backends. Regex rules are the production default. They have the form `KIND:TAG=REGEX`, are evaluated in order (first match wins), and an AI agent can refine the rules with AgentProf until the unmatched rate drops below 5%, typically in 5–10 rounds. A local LLM tagger, such as a quantized 3B-parameter model running through llama.cpp (ggml-org 2023), generates one-word tags with grammar-constrained decoding that forces output to match a predefined token grammar. Caching lets an AI agent iterate the prompt as part of the configuration. Unsupervised TF-IDF (Salton and Buckley 1988) and K-Means (MacQueen 1967) clustering groups prompts without predefined rules to guide rule authoring.

Profile outputs. Output formats include pprof protobuf (compatible with `go tool pprof -http`), folded-stack text (compatible with `flamegraph.pl`), standalone SVG flame graphs, and JSON summaries.

5 Evaluation

Evaluation questions. We ask four fixed questions: (1) whether semantic profiling improves resource attribution (RQ1), (2) whether profiler output corresponds to real problems (RQ2), (3) how accurate its semantic tags are (RQ3), and (4) what complete profiling costs (RQ4). Together, they test the attribution and localization benefits, the semantic fields on which they depend, and the practicality of profiling.

Datasets and workloads. We use two classes of data. The first class comprises *real agent trajectories*, including 325 readable local trajectories (198 Codex, 50 Claude, 77 Claude subagent) from a multi-month development project and containing 183,714 system observations. The second comprises *public annotated trajectories*, including 15 families of real agent or human executions that mapping rules convert into 47,590 operations. These cover web agents (WebLINX (Lù, Kasner, and Reddy 2024), Mind2Web (Deng et al. 2023),

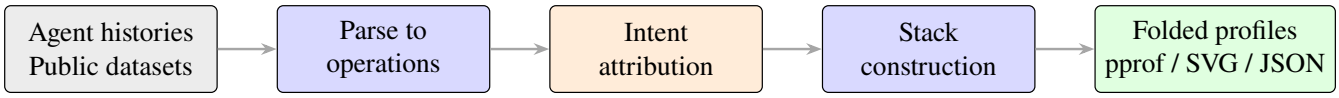

Figure 2: Data-flow and algorithm pipeline. Local agent histories enter through local-history parsing, whereas public datasets enter as preconstructed operation records. Both paths produce uniform operations. Intent attribution, stack construction, and folding are query-time algorithms applied identically to both sources.

AgentTrek (Xu et al. 2024), WebShop (Yao et al. 2022)), API agents (API-Bank (Li et al. 2023), ToolBench (Qin et al. 2024), tau-bench (Yao et al. 2024)), coding agents (SWE-agent (Yang et al. 2024)), and mobile/GUI agents (AndroidCtrl (Li et al. 2024), GUI-Odyssey (Lu et al. 2025), ScaleCUA (Liu et al. 2025)). Four families with ground-truth quality annotations (AgentRewardBench (Lù et al. 2025), SATraj-OS (Chen et al. 2026), AgentNet (Wang et al. 2025), OSWorld-Human (WukLab 2025)) provide hidden annotations for RQ2.

5.1 RQ1: Resource Attribution

Operation-stack field ablation. RQ1 asks whether semantic profiling improves resource attribution. Here, semantic tags are low-cardinality fields derived from natural-language behavior. The `prompt_tag` field is semantic, whereas `session` is a source identifier. We test whether the semantic field improves resource attribution beyond what traditional tag-free tools provide on 325 real Codex/Claude trajectories (183,714 system observations). We compare four operation-stack field selections, namely neither field, `session` only, `prompt_tag` only, and both. We measure the mixed-weight percentage, the fraction of system-effect cost in groups that mix observations from multiple task categories. A lower mixed-weight percentage means the profiler correctly separates effects by task category. The residual percentage measures weight in groups where the majority category accounts for less than 90% of the group, capturing partial mixing.

Figure 3 shows that without the `prompt_tag` field, nearly all system-effect cost falls into mixed groups. For example, a flat `cargo test` counter shows 2,903 executions but cannot distinguish whether they came from `review`, `debug`, `refactor`, or `test` prompts. The `prompt_tag` field is decisive. Adding it drops both mixed weight and residual by more than half and nearly doubles the number of unique stacks because it separates observations that share the same system call but originate from different task intents. The `session` field alone barely helps because trajectories typically contain multiple intent phases.

A tag-permutation test (1,000 shuffles, $p = 0.001$) confirms the separation is not random. Semantic-tag quality is validated in RQ3. The operation-stack field ablation confirms that traditional tag-free tools cannot separate system effects by task intent. The `prompt_tag` field is necessary for meaningful resource attribution.

Field-selection comparison. Beyond tag separation, we test whether different operation-stack fields produce views that flat grouping cannot. We take the same 13,265 operations from nine public datasets and fold them with five ordered field lists: (1) `project`, `dataset`; (2) those fields followed by `task`, `phase`; (3) those fields followed by `op`, `tool`, `status`; (4) the third list with `action` before `status`; and (5) the fourth list with `session` after `dataset`. The same operations fold into 9 dataset groups, 57 phase groups, 226 semantic groups, 455 action groups, or 3,757 per-session groups by changing only the stack fields. A flat `GROUP BY` on one tag cannot produce these views because dataset-level budgets, action-level duplication, and per-session detail require different stack depths. The same model also covers all 15 public trajectory families without type-specific objects. The field-selection comparison shows that operation stacks provide a multi-resolution view unavailable from flat grouping. One model answers questions at dataset, phase, action, and session granularity by changing only the field selection.

Weight-function comparison. A profiler restricted to token count might miss bottlenecks visible under another metric. We test whether different weight functions produce redundant or distinct rankings on the same trajectories. The `time` and `tokens` views of the same 325 trajectories share only 7/10 top prompt categories (Spearman $\rho = 0.623$). `network`-tagged prompts rank 8th by time (I/O wait) but 93rd by tokens. The rankings are not redundant. A single weight function would miss bottlenecks that are prominent under a different metric, confirming that multi-view profiling is necessary.

Figure 1 shows the resulting flame graphs for the `tokens`, `time`, and `files` views, illustrating that changing only the stack fields and weight function produces different aggregates from the same operations.

Automatic stack induction. Finally, we test whether the profiler can produce useful groups without any user-specified fields. We run automatic stack induction on the six RQ2 tasks and compare its group count and localization quality with flat summaries and per-session grouping. Induced stacks produce a median of 12 groups (versus 285 for per-session and 1 for flat), with 4/6 tasks generating variable-depth stacks. Without user configuration, induced stacks reduce inspection work to 65.3% of flat summaries and achieve median average precision (AP, the area under the precision-recall curve for the visible-field group ranking) of 0.276 using only visible fields. Hand-specified stacks reach median AP 0.312. Automatic induction therefore provides a useful zero-configuration baseline that substantially outperforms flat summaries.

Together, these results show that semantic tags separate measured effects, query-time fields and weights expose distinct attribution views, and automatic induction supplies the existing zero-configuration alternative.

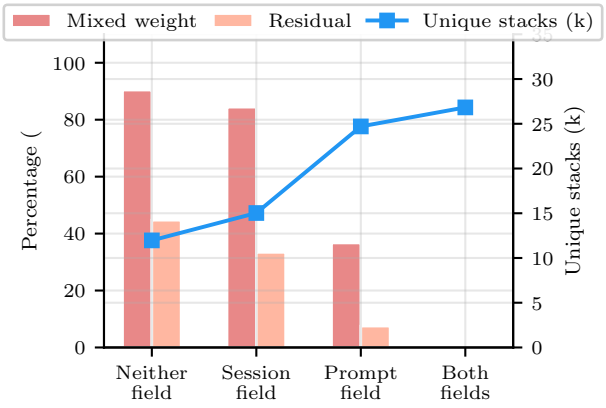

Figure 3: RQ1 operation-stack field ablation over 183,714 observations from 325 real trajectories. Bars show mixed weight and residual (left axis). The line shows unique stacks (right). Adding `prompt_tag` cuts mixed weight from 90.4% to 36.7% while increasing stacks from 12,000 to 25,000. The `session` field alone barely helps (84.4%).

5.2 RQ2: Real-Problem Localization

RQ2 asks whether profiler output corresponds to real problems.

Benchmark and hidden annotations. We test whether the profiler's top-ranked groups contain real failures, safety violations, or wasted effort. We use public benchmark datasets with ground-truth annotations for failure, safety violations, redundant steps, and human-task boundaries. We hide the annotations from the profiler and check whether its top-ranked groups contain the corresponding positives, analogous to injecting known bottlenecks in a CPU profiler evaluation.

The benchmark comprises six tasks across four datasets, totaling 34,539 task-operation instances and 3,699 positives. The tasks cover AgentRewardBench looping and side effects, SATraj-OS unsafe operations, AgentNet incorrect and redundant steps, and OSWorld-Human group starts.

Baselines. We compare seven grouping methods. The profiler ranks the five non-oracle methods using only visible fields, and hidden annotations score the resulting order after ranking. A flat summary places all operations in one group, while per-session grouping uses session boundaries. Native hierarchy uses each dataset's own structure, and raw-action grouping uses the unprocessed action- type field. Operation-stack profiling applies the model from §3. One oracle preserves operation-stack groups and ranks them by hidden-positive count. The other also uses hidden annotation fields to form groups.

Localization results. Table 1 shows that flat summaries recover all positives but require inspecting everything (Work@5 = 1.0). Per-session grouping finds a first positive quickly (WTFP = 0.004) but fragments the workload into 285 groups with only 2.3% top-five recall. Operation-stack profiling inspects 9.4% of the work in the top five groups, recovers 39.0% of positives within 30% of the inspection budget, and reduces the median group count to 157.5.

Statistical validation. Across 10,000 task-family bootstrap resamples, the 95% intervals for operation-stack AP differences versus flat and width-only operation stacks exclude zero. In a separate 2,000-trial label-permutation control that fixes each visible ranking and reallocates positives within same-size groups, operation-stack query-aware AP exceeds its null on all six tasks.

Stack sensitivity. Stack depth affects localization quality. Sweeping the induced-stack depth cap from 1 to 5 shows that AP peaks at depth 3 (median 0.287), while different tasks achieve their best AP at depths 2–5, so depth should be tuned per objective. Modifying stack fields, mapping rules, or ranking criteria and re-running on the same operations improves AP on 5/6 tasks (median improvement 0.038), confirming that these configuration choices improve localization.

Inspection tradeoffs. Flat, per-session, and operation-stack profiling expose an inspection-effort/ coverage tradeoff. Operation-stack profiling concentrates positives into fewer, higher-quality groups, providing broader coverage at lower total effort than flat summaries and less fragmentation than per-session grouping. Flat summaries find every positive only after full inspection, whereas per-session grouping reaches a first positive quickly but scatters the rest across hundreds of groups.

Answer. The comparison also serves as evidence for the semantic operation stack model (§3). Per-session grouping and native hierarchy are alternative abstractions that fix the hierarchy at session-specific or dataset-native boundaries. Operation stacks subsume both as special cases. Selecting `session` reproduces per-session grouping, while selecting the dataset's corresponding ordered native fields (e.g., episode then step) reproduces its native hierarchy. On localization, operation stacks improve top-five recall over per-session grouping on 5/6 tasks, reduce median group count by 45%, and save 90% of inspection work versus flat summaries. Per-session grouping retains lower work-to-first-positive on 4/6 tasks, confirming that profilers and debuggers are complementary.

5.3 RQ3: Tag Accuracy

RQ3 asks how accurately semantic tags derived by intent attribution or mapping rules reproduce benchmark ground-truth annotations.

Public benchmark datasets provide native action annotations as ground truth for mapping quality. For each of nine datasets, we infer mapping rules from the other eight. We then compare the mapping-derived `phase` field with native `action` annotations using V-measure (Rosenberg and Hirschberg 2007), a clustering-agreement score from 0 to 1. We also compute boundary F1 between mapped phase transitions and native action boundaries.

Figure 4 reports both metrics where applicable. V-measure quantifies whether the mapping groups operations into the same clusters as the ground-truth annotations. Boundary F1 measures whether phase-transition points align. Seven of

Method	Hidden	AP	R@5	F1@5	Work@5	R@30%	WTFP	Groups
Flat (all-in-one)	0	0.168	1.000	0.287	1.000	0.000	1.000	1.0
Per-session grouping	0	0.348	0.023	0.043	0.016	0.356	0.004	285.0
Native hierarchy	0	0.357	0.867	0.282	0.854	0.338	0.058	—
Raw-action grouping	0	0.274	0.244	0.220	0.151	0.333	0.037	—
Operation-stack profiling	0	0.312	0.188	0.198	0.094	0.390	0.038	157.5
Operation stack + oracle ranker	1	0.599	0.300	0.403	0.044	0.564	0.012	—
Hidden-annotation grouping	1	1.000	0.670	0.797	0.115	1.000	0.038	—

Table 1: RQ2 hidden-annotation localization (medians over six tasks). AP is average precision. R@5 and F1@5 are recall and F1 in the top five groups. Work@5 is their work fraction. R@30% is recall at 30% inspection. WTFP is work to first positive. Hidden=1 denotes annotation-based grouping or ranking. "—" denotes task-dependent group count.

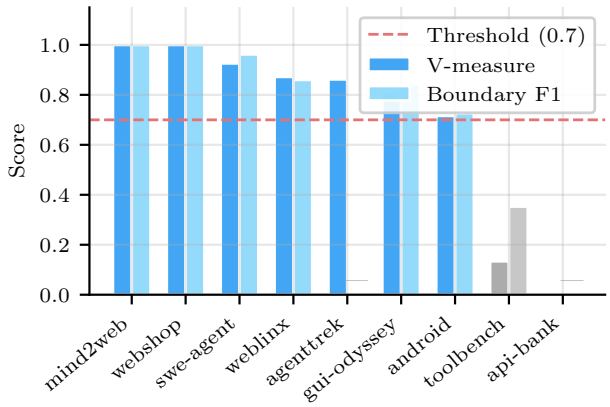

Figure 4: RQ3 leave-dataset-out mapping quality over 13,265 operations from nine datasets. Bars show V-measure between mapped `phase` and native `action`, and boundary F1 for phase transitions. "—" marks N/A. Seven of nine datasets exceed 0.7 on every applicable metric.

nine datasets exceed 0.7 on every applicable metric. Rules inferred from other families generalize by assigning operations to the correct semantic phase and, where native boundaries exist, detecting transitions accurately.

Answer. Under leave-dataset-out evaluation, mapping-derived phases exceed 0.7 on every applicable metric for seven of nine datasets.

5.4 RQ4: Profiling Cost

RQ4 asks what complete profiling costs.

Scope. AgentProf runs offline after a session, so profiling adds no agent-runtime overhead. Its cost is post-hoc time and resources. A run selects stack fields, a predicate, weight and ranking functions, then parses, enriches, constructs stacks, and folds the operations.

End-to-end construction. Across 76 configurations, median end-to-end time is 1.6 s (p95 2.8 s, max 4.3 s). The slowest runs process views with over 600 unique stacks, while runs that vary only ranking criteria or stack depth all complete within 1.6 s.

Intent-attribution cache. Intent attribution adds a one-time cost for new prompts. Of 118,021 tag requests on 325 real trajectories, 70% were cache hits. The full-history run issued 35,136 llama.cpp calls. Separately, a local 3B model achieved 31 ms p95 latency per tag on 300 fragments, and grammar-constrained decoding produced valid output for 2,700/2,700 tags across three model configurations. Because tags are cached, re-profiling the same trajectories with different stack fields or predicates incurs no additional LLM calls.

Thus, complete profiling takes seconds on these workloads, while cached re-profiling changes the view without further tagger calls.

6 Related Work

Agent-observability systems center on per-execution tracing, evaluation, and metadata aggregation (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024; Arize AI 2024b; Dong, Lu, and Zhu 2024; Balusu 2026; Wang et al. 2026b). Datadog LLM Observability (Datadog 2025) and Laminar (Laminar 2025) also aggregate cost or metadata, cluster inputs, and extract structured signals. AgenTRx (Barke et al. 2026), TELBench (Wang et al. 2026a), AgentAtlas (Mazaheri and Mazaheri 2026), TrajAD (Liu et al. 2026), and AgentFixer (Mulian et al. 2026) analyze or localize faults within trajectories, whereas AgentProf models cross-layer activities as operations and constructs query-time operation stacks for profiling across trajectories.

7 Conclusion

Agent observability needs profiling, not only debugging. AgentProf realizes this thesis through semantic operation stacks and pluggable intent attribution and stack construction. Across the four RQs, semantic profiling separates over 90% of system-effect cost that per-session views leave mixed, locates annotated problems while inspecting 9.4% of the work required by flat summaries, and produces tags that agree with ground truth on seven of nine held-out datasets. It constructs the reported profiles in median 1.6 s across 76 configurations.

References

Anthropic. 2025. Claude Code: An Agentic Coding Tool. https://code.claude.com/docs/en/overview.

Arize AI. 2024a. Arize Phoenix. https://arize.com/docs/phoenix.

Arize AI. 2024b. OpenInference Specification. https://arize-ai.github.io/openinference/spec/.

Balusu, K. C. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*.

Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.

Chen, X.; Yin, Z.; He, S.; Huang, B.; Lei, S.; et al. 2026. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230.

Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.

Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Dong, L.; Lu, Q.; and Zhu, L. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285.

ggml-org. 2023. llama.cpp: LLM Inference in C/C++. https://github.com/ggml-org/llama.cpp.

Google. 2024a. Perfetto Trace Processor. https://perfetto.dev/docs/analysis/trace-processor.

Google. 2024b. pprof. https://github.com/google/pprof.

Gregg, B. 2011. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.

Laminar. 2025. Signals: Structured Event Extraction from Traces. https://docs.lmnr.ai/signals/introduction.

LangChain. 2024. LangSmith Observability. https://docs.langchain.com/langsmith/observability.

Langfuse. 2024. Langfuse Observability. https://langfuse.com/docs/observability/overview.

Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 3102–3116.

Li, W.; Bishop, W. E.; Li, A.; Rawles, C.; Campbell-Ajala, F.; Tyamagundlu, D.; and Riva, O. 2024. On the Effects of Data Scale on UI Control Agents. arXiv preprint arXiv:2406.03679.

Liu, Y.; Zhang, C.; Han, Z.; Liu, H.; Wang, Y.; Yu, Y.; Wang, X.; and Yin, Y. 2026. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. arXiv preprint arXiv:2602.06443.

Liu, Z.; Xie, J.; Ding, Z.; Li, Z.; Yang, B.; et al. 2025. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. arXiv preprint arXiv:2509.15221.

Lu, Q.; Shao, W.; Liu, Z.; Meng, F.; Li, B.; Chen, B.; Huang, S.; Zhang, K.; Qiao, Y.; and Luo, P. 2025. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

Lù, X. H.; Kasner, Z.; and Reddy, S. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. arXiv preprint arXiv:2402.05930.

Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. arXiv preprint arXiv:2504.08942.

MacQueen, J. 1967. Some Methods for Classification and Analysis of Multivariate Observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics*, 281–297. Berkeley, California: University of California Press.

Mazaheri, P.; and Mazaheri, K. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530.

Mulian, H.; Zeltyn, S.; Levy, I.; Galanti, L.; Yaeli, A.; and Shlomov, S. 2026. AgentFixer: From Failure Detection to Fix Recommendations in LLM Agentic Systems. Accepted at the 1st International Workshop on Agentic Engineering (AGENT '26, co-located with ICSE); arXiv preprint arXiv:2603.29848.

OpenAI. 2025a. Improving Support with Every Interaction at OpenAI. https://openai.com/index/openai-support-model/.

OpenAI. 2025b. Introducing Codex. https://openai.com/index/introducing-codex/.

OpenAI. 2026. Introducing the Codex App. https://openai.com/index/introducing-the-codex-app/.

OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. https://github.com/open-telemetry/semantic-conventions-genai.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerber, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

Rosenberg, A.; and Hirschberg, J. 2007. V-Measure: A Conditional Entropy-Based External Cluster Evaluation Measure. In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, 410–420. Prague, Czech Republic: Association for Computational Linguistics.

Salton, G.; and Buckley, C. 1988. Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing & Management*, 24(5): 513–523.

Wang, J.; Feng, Z.; Wu, J.; Li, R.; Xie, Q.; Ren, Y.; Zhu, H.; Han, X.; Meng, F.; Feng, J.; and Liu, J. 2026a. Where Do Deep-Research Agents Go Wrong? Span-Level

Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060.

Wang, X.; Wang, B.; Lu, D.; Yang, J.; Xie, T.; et al. 2025. AgentNet. https://huggingface.co/datasets/xlangai/AgentNet.

Wang, Y.; Zhang, J.; Cai, T.; Liu, Z.; Sun, Q.; Sun, Z.; Wu, Z.; Dong, M.; Zheng, M.; Yin, X.; and Zhu, Y. 2026b. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. arXiv preprint arXiv:2606.04990.

WukLab. 2025. OSWorld-Human. https://github.com/WukLab/osworld-human.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.

Xu, Y.; Lu, D.; Shen, Z.; Wang, J.; Wang, Z.; Mao, Y.; Xiong, C.; and Yu, T. 2024. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. arXiv preprint arXiv:2412.09605.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems (PACMI '25, co-located with SOSP)*.